

Hugging Face Transformers Tutorial

Generative Algorithms for Sound and Music



Universitat
Pompeu Fabra
Barcelona

MTG
Music Technology
Group

Agenda

Part 1:

- Brief intro to HF
- Tutorial overview of Hugging Face Transformers library
- Live demo: symbolic music generation in the style of 

Part 2:

- Code walkthrough
- LORA as a fine tuning technique
- Deployment (if we have time)

Why are we teaching this class?

(by “we” I mean... me, mostly)

- This is how researchers actually work now

Why are we teaching this class?

(by “we” I mean... me, mostly)

- This is how researchers actually work now
- We've broken enough generation systems to learn from it

Why are we teaching this class?

(by “we” I mean... me, mostly)

- This is how researchers actually work now
- We've broken enough generation systems to learn from it
- We've deployed systems that actually worked on other people's machines.

Why are we teaching this class?

(by “we” I mean... me, mostly)

- This is how researchers actually work now
- We've broken enough generation systems to learn from it
- We've deployed systems that actually worked on other people's machines
- We wish someone had taught us this last year

Why are we teaching this class?

(by “we” I mean... me, mostly)

- This is how researchers actually work now
- We've broken enough generation systems to learn from it
- We've deployed systems that actually worked on other people's machines.
- We wish someone had taught us this last year.
- We love u <3

Why are we teaching this class?

(by “we” I mean... me, mostly)

- This is how researchers actually work now
- We've broken enough generation systems to learn from it
- We've deployed systems that actually worked on other people's machines.
- We wish someone had taught us this last year.
- We love u <3
- Disclaimer: we are just TAs... we're learning too!

Hugging Face



What is Hugging Face?

- Ecosystem for building, sharing and deploying ML models
- [Platform](#) + [Libraries](#)

What is Hugging Face?

Github for AI models

The HF ecosystem - what you need to know

- [Model Hub](#) - where pretrained models live (systems like midi-gpt2)

The HF ecosystem - what you need to know

- [Model Hub](#) - where pretrained models live (systems like midi-gpt2)
- [Datasets](#) - training data ready to use

The HF ecosystem - what you need to know

- [Model Hub](#) - where pretrained models live (systems like midi-gpt2)
- [Datasets](#) - training data ready to use
- [Spaces](#) - deploy demos (more of this later)

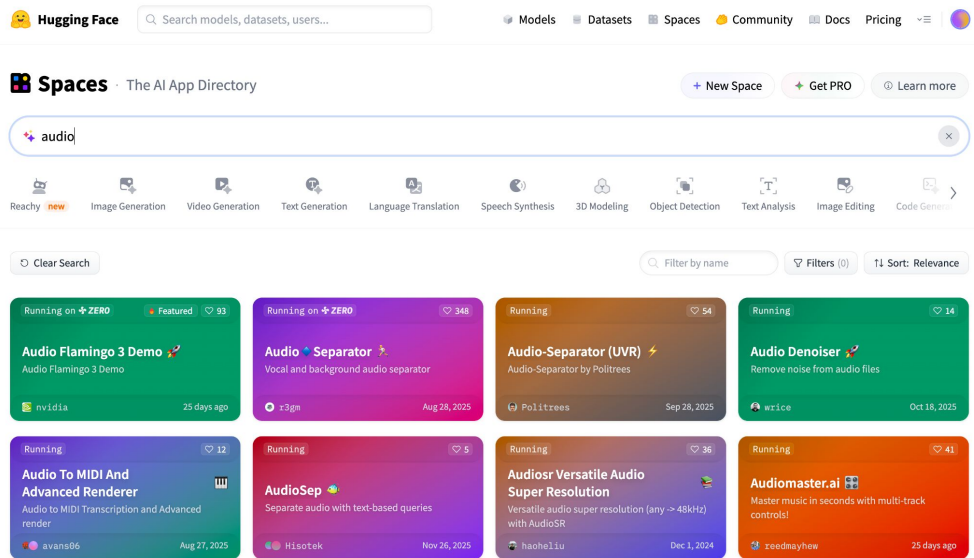
The HF ecosystem - what you need to know

- [Model Hub](#) - where pretrained models live (systems like midi-gpt2)
- [Datasets](#) - training data ready to use
- [Spaces](#) - deploy demos (more of this later)
- [Transformers Library](#) - High-level API for state-of-the-art models (text, audio, vision).

HF in the context of audio-music technology

By the community

- Text-to-Speech and Voice cloning: [XTTS](#), [XTTS v2](#)
- Symbolic Music Generation [midi-gpt2](#) ([github](#))





Transformers crash course

- [Pipeline](#)
- [Trainer](#)
- [Generate](#)
- Check [official tutorial](#)

First Hands-on with the 🤗 Transformers Library

- Check out this [notebook](#) and follow along!

Congrats! You just

- Installed the HFT Library
- Downloaded models locally for a certain task
- Made batch inference

Now you can add to the CV: Proficiency in HF ecosystem

That was cute, but

**gentlement lets
get serious**



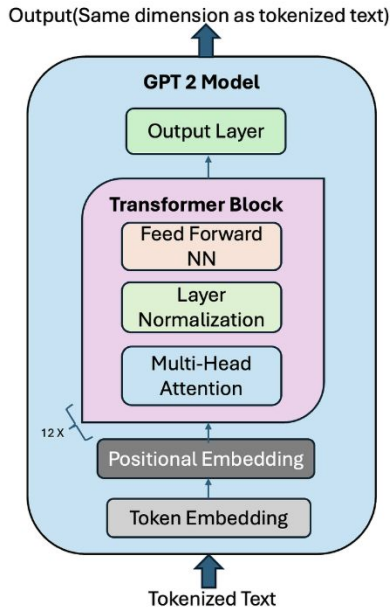
Project idea!

Model: [midi-gpt2](#)

- Symbolic music generation model: [video demo](#)

Model: [midi-gpt2](#)

- Symbolic music generation model: [video demo](#)
- Uses GPT-2's architecture (OpenAI's design) but trained from scratch on MIDI data, not text.
 - Decoder-Only Transformer
 - Autoregressive

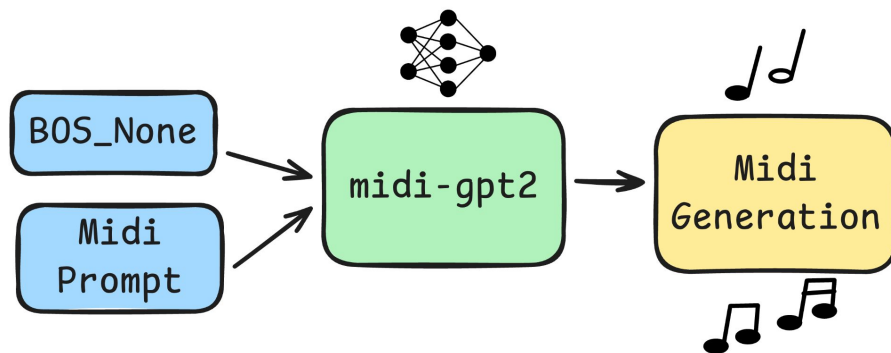


Model: [midi-gpt2](#)

- Symbolic music generation model: [video demo](#)
- Uses GPT-2's architecture (OpenAI's design) but trained from scratch on MIDI data, not text.
- [model](#) and [tokenizer](#) deployed to 🙌

Model: [midi-gpt2](#)

- Symbolic music generation model: [video demo](#)
- Uses GPT-2's architecture (OpenAI's design) but trained from scratch on MIDI data, not text.**
- [model](#) and [tokenizer](#) deployed to 🙌
- Generation with midi prompt or **BOS_None**



** There are several models on the project. GPT2 is the one deployed in HF

All good, but

Original dataset unknown, no paper and no docs 🙄

```
data/
├── chunks_train_2/    # Training chunks (MIDI-GPT2 trained on this)
│   └── *.mid          # Segmented MIDI files
├── chunks_val/        # Validation chunks
│   └── *.mid
├── wikifonia_midi/    # Used for Aria harmonization
└── chopin/            # Used for Aria style transfer
```



The problem(?)

Results sound pretty good, but:

The problem(?)

Results sound pretty good, but:

- Feels a bit boring

The problem(?)

Results sound pretty good, but:

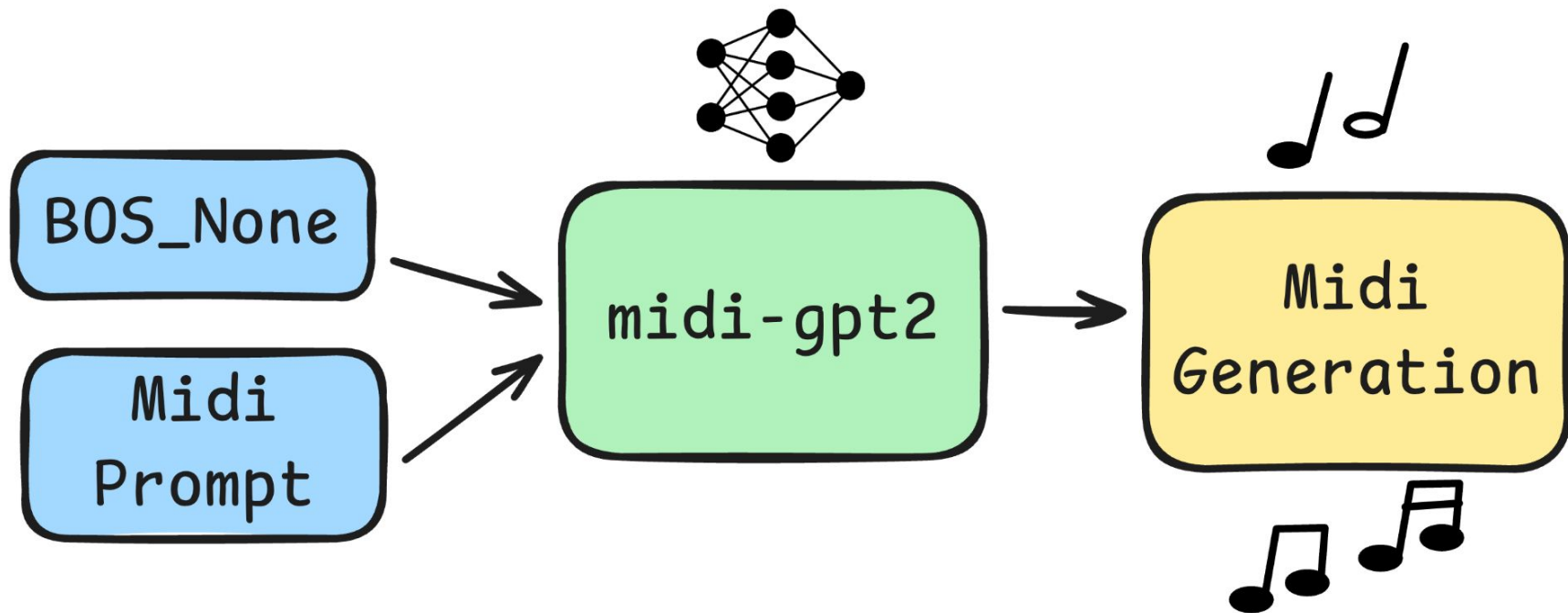
- Feels a bit boring
- Tends to follow a simple formula: Chord + Arpeggio

The problem(?)

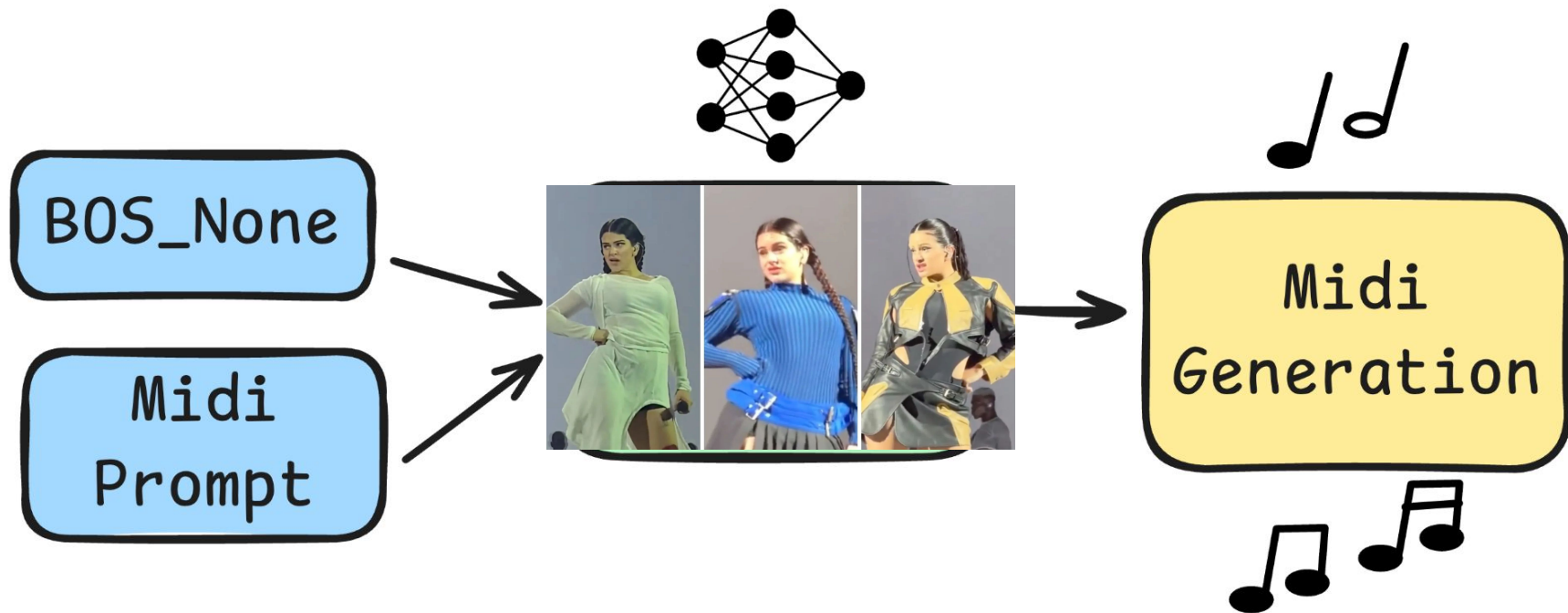
Results sound pretty good, but:

- Feels a bit boring
- Tends to follow a simple formula: Chord + Arpeggio
- Not that interesting?

So... what if...



So... what if...



lux-midi-gpt2

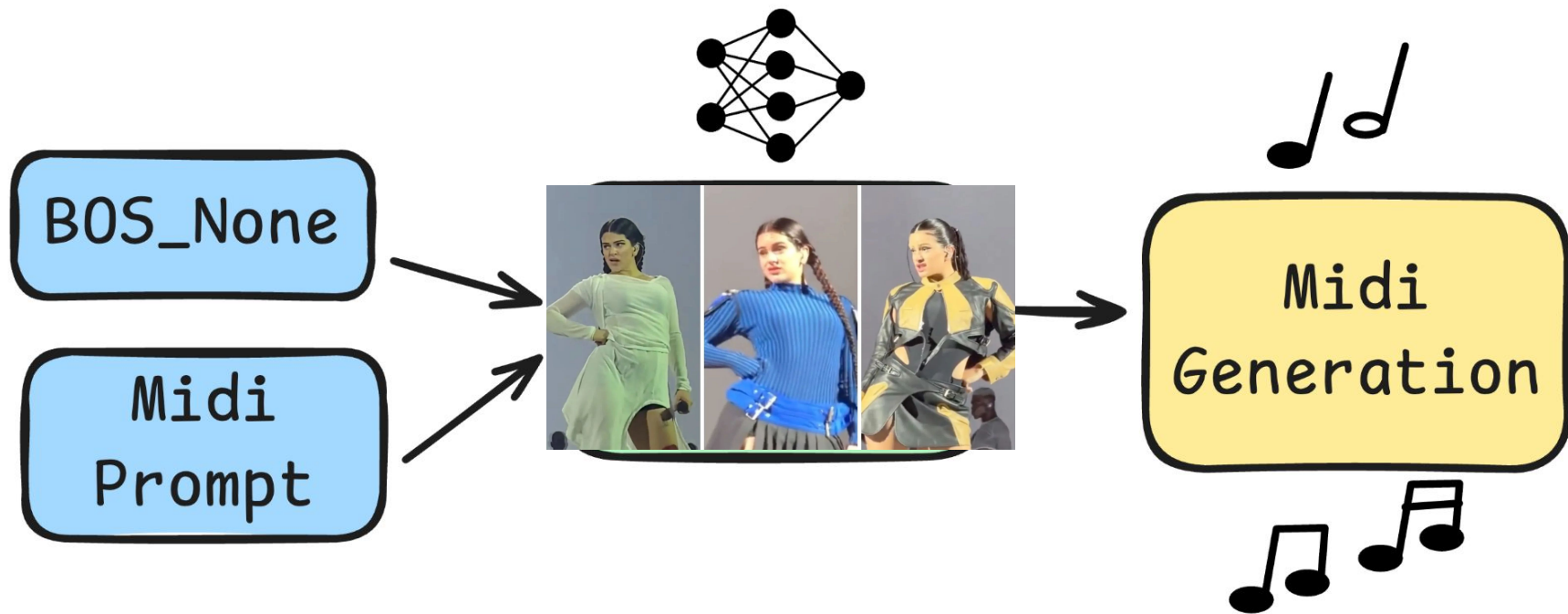
- We trained a LoRA on Rosalía's latest album (HFT + [PEFT](#))

**I TRAINED A LORA
ON ROSALÍA'S LATEST ALBUM
FOR MY GENERATIVE AI MUSIC CLASS**





So, how did we do it?



LoRA intuition

- Freezes the original model weights and injects small, trainable low-rank matrices into specific layers.

LoRA intuition

- Freezes the original model weights and injects small, trainable low-rank matrices into specific layers.

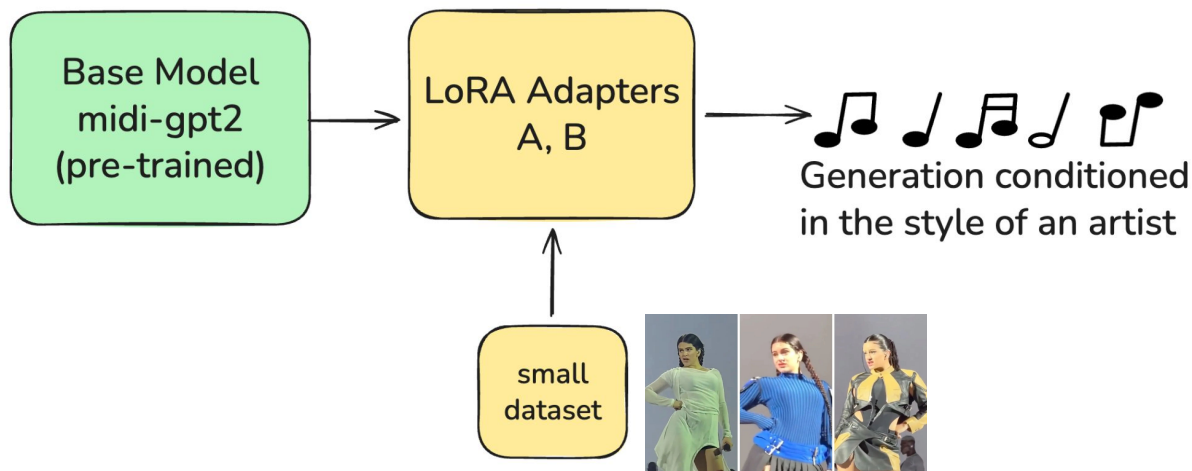
Where:

$$\Delta W = B \times A$$

- $W \in \mathbb{R}^{d \times k}$ — original frozen weights
- $A \in \mathbb{R}^{r \times k}$ — trainable (small)
- $B \in \mathbb{R}^{d \times r}$ — trainable (small)
- $r \ll \min(d, k)$ — the rank (often 4–64)

LoRA intuition

- Freezes the original model weights and injects small, trainable low-rank matrices into specific layers.

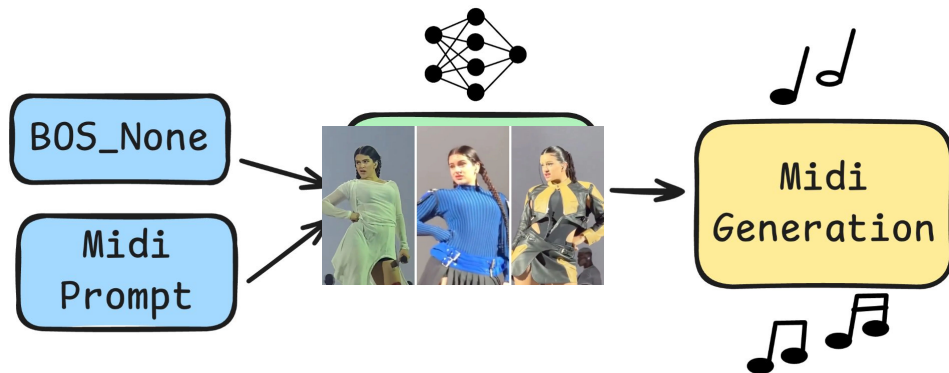


More info about LoRA

- Check these tutorials on [medium](#) and [DataCamp](#)

So, how did we do it?

- Dataset: collection, augmentation
- Define a LoRA adapter
- Train! (a lot of attempts)
- Test/Audit results
- Deployment



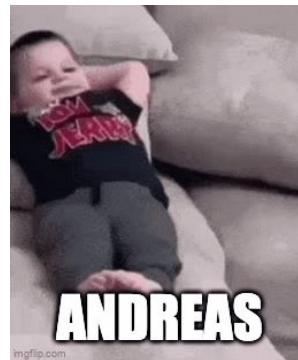
Data was not easy to get

- MuseScore hub
- Andreas Expertise and refinement
- MIDI Performance vs Score



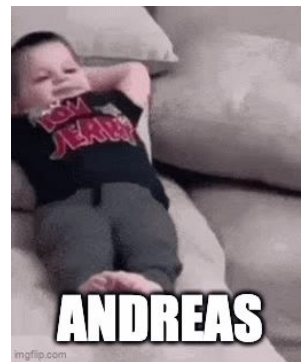
Data segmentation

- Separated MIDI files by song segments: Intro, Verse, Chorus
- From 15 files to 111 files



LoRA definition

```
lora_config = LoraConfig(  
    r=args.lora_r,  
    lora_alpha=args.lora_alpha,  
    lora_dropout=args.lora_dropout,  
    bias="none",  
    task_type=TaskType.CAUSAL_LM,  
    target_modules=["c_attn", "c_proj", "c_fc"],  
)  
model = get_peft_model(model, lora_config)
```



LoRA target modules

- GPT-2 layer names
 - *c_attn*: Self-attention projection
 - *c_proj*: Output projection
 - *c_fc*: Feed-forward network
 - *r*=16: Standard rank for LoRA
 - *lora_alpha*=32: 2x rank (common practice)

We tried a lot of configurations

outputs > lora-midi-gpt2-lux-segmented-bz2-gas16-e5-wd0_1 > ≡ logs.log

```
1 python src/train_lora.py \  
2   --data-dir ~/datasets/lux_midi/segments \  
3   --output-dir outputs/lora-midi-gpt2-lux-segmented-bz2-gas16-e5-wd0_1 \  
4   --batch-size 2 \  
5   --grad-accum-steps 16 \  
6   --epochs 5 \  
7   --weight-decay 0.1 \  
8   --val-split 0.05
```

▼ outputs

- > lora-midi-gpt2-lux-bz4-gas4-e8
- > lora-midi-gpt2-lux-segmented-bz2-gas8-e4
- > lora-midi-gpt2-lux-segmented-bz2-gas16-e3-wd0_1
- > lora-midi-gpt2-lux-segmented-bz2-gas16-e5-wd0_1
- > lora-midi-gpt2-lux-segmented-bz2-gas16-e10
- > lora-midi-gpt2-lux-segmented-bz4-gas4-e8
- > lora-midi-gpt2-lux-segmented-bz4-gas16-e3
- > lora-midi-gpt2-lux-segmented-bz6-gas32-e4-wd0_5

And listened to a lot of generations

> `ls generations`

```
bz2gas16e10_segmented_berghain1__bad.mid  
bz2gas16e10_segmented_perla1__bad.mid  
bz2gas16e10_segmented_reliquia1__bad.mid  
bz2gas16e3_segmented_perla1__good.mid  
bz2gas16e3wd01_segmented_berghain_1__good.mid  
bz2gas16e3wd01_segmented_perla1__meh.mid  
bz2gas16e3wd01_segmented_reliquia_1__bad.mid  
bz2gas16e3wd01_segmented_reliquia_2__bad_meh.mid  
bz2gas16e5wd01_segmented_berghain_1__bad_meh.mid  
bz2gas16e5wd01_segmented_berghain_2__bad_meh.mid  
bz2gas16e5wd01_segmented_berghain_3__meh.mid  
bz2gas16e5wd01_segmented_perla1__meh.mid  
bz2gas16e5wd01_segmented_reliquia_1__bad.mid  
bz2gas8e4_segmented_berghain_1__meh.mid  
bz2gas8e4_segmented_reliquia_1__bad.mid  
bz4gas16e3_segmented_berghain1__good_great.mid
```

```
bz4gas16e3_segmented_perla1__good.mid  
bz4gas16e3_segmented_reliquia1__bad.mid  
bz4gas32e4wd05_segmented_berghain_1__interesting.mid  
bz4gas32e4wd05_segmented_reliquia_1__bad.mid  
bz4gas4e8_output1_berghain__good.mid  
bz4gas4e8_output1_empty__bad_but_interesting.mid  
bz4gas4e8_output1_reliquia__good.mid  
bz4gas4e8_perla1__good.mid  
bz4gas4e8_segmented_output1_berghain__bad.mid  
bz4gas4e8_segmented_output1_empty__bad.mid  
bz4gas4e8_segmented_output1_reliquia__bad.mid  
bz4gas4e8_segmented_output2_berghain__bad.mid  
bz4gas4e8_segmented_perla1__bad.mid  
midigpt2_berghain1.mid  
midigpt2_perla1.mid  
_ midigpt2_reliquia1.mid
```

When to stop?

- No correct set of parameters/configurations
- Keep trying until it sounds good to you



Deployment

We can't deploy the model for legal reasons

Deployment

We can't deploy the model for legal reasons
but...

Deployment

- Deploy with HF [push_to_hub](#)
- Deploy as private HF space for testing
- Use [secrets](#) and [env vars](#)

Share and show your work!

- Make a gradio app: like this (screenshot)
- Add it to your portfolio
- Share and collaborate :)

Thanks :)